MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD

DEPARTMENT OF MECHANICAL ENGINEERING

AUTOMATIC PLANT IRRIGATION SYSTEM

IoT-Based Soil Moisture Monitoring and Remote Manual Pump Control Using NodeMCU ESP8266 and Blynk IoT Cloud



Conceptual illustration only; it is not a photograph of the final assembled prototype.

FINAL PROJECT REPORT

Submitted by

Anuj Parihar	**20247015**
Aman Kharwar	20247010
Mayank Gupta	20247027
Sudarshan Jha	20247051

Project Supervisor: Dr. Mandeep Dhanda

Academic Session: 2026

PRELIMINARY PAGES

Student declaration

Declaration for project submission

We, the undersigned students of the Department of Mechanical Engineering, Motilal Nehru National Institute of Technology Allahabad, submit this report titled “Automatic Plant Irrigation System” as an account of the project work carried out by our team during the academic session 2026.

We declare that this report has been prepared to describe the project architecture, hardware selection, embedded firmware, Blynk IoT Cloud integration, calibration approach, testing methodology, limitations and proposed future work. Wherever external technical documentation has been used, it has been cited in the references.

The report distinguishes the functionality present in the submitted firmware from proposed extensions. In particular, the current firmware automatically measures and reports soil moisture, while pump ON/OFF operation is initiated manually through Blynk virtual pin V1. Automatic threshold-based pump control is not claimed as an implemented feature.

We further confirm that credentials such as Wi-Fi passwords and device authentication tokens have been removed from this public report. Only placeholders are shown in the code appendix.

Student details

Name	Registration number
Anuj Parihar	20247015
Aman Kharwar	20247010
Mayank Gupta	20247027
Sudarshan Jha	20247051

PRELIMINARY PAGES

Acknowledgement

We express our sincere gratitude to our project supervisor, Dr. Mandeep Dhanda, for the academic guidance and direction provided during the development and documentation of this project. The process of translating an IoT concept into a clear system architecture required us to examine the relationships between sensing, embedded control, cloud communication and electrical actuation.

We also acknowledge the Department of Mechanical Engineering at Motilal Nehru National Institute of Technology Allahabad for providing the academic setting in which the project could be planned, discussed and documented. The project encouraged us to connect mechanical applications such as irrigation with electronics, programming and networked systems.

Finally, we thank one another as team members for contributing to the analysis, firmware, platform configuration, hardware planning, technical reporting and presentation work. The report records the current state of the implementation honestly and identifies the measurements and validation activities that remain necessary before the system can be treated as a field-ready irrigation controller.

PRELIMINARY PAGES

Abstract

This project presents an Internet of Things (IoT)-enabled plant-watering prototype based on the NodeMCU ESP8266, an analog soil-moisture sensor, an active-LOW relay module, a DC water pump and the Blynk IoT Cloud platform. The system measures the analog output of the soil-moisture sensor at pin A0, converts the raw reading into a bounded percentage-like indicator using two calibration endpoints, and publishes the result to Blynk virtual pin V0. A user can operate the pump remotely through a dashboard control mapped to virtual pin V1.

The submitted firmware uses the calibration constants DRY_VALUE = 1023 and WET_VALUE = 450. The sensor routine is scheduled at an interval of 1000 milliseconds using BlynkTimer. The relay output is assigned to D4 and follows active-LOW logic: LOW energizes the relay and HIGH de-energizes it. The program commands the pump OFF during startup and again when Blynk reports a successful connection.

The current implementation demonstrates remote soil-moisture monitoring and manual pump actuation. It does not automatically switch the pump according to the moisture reading. The displayed percentage is a linear indicator derived from the selected endpoints and is not a laboratory-validated volumetric water-content measurement. Calibration, final electrical verification and integrated pump testing are therefore treated as required validation activities.

Keywords

Internet of Things; NodeMCU ESP8266; Blynk IoT; soil-moisture sensing; analog-to-digital conversion; virtual pins; active-LOW relay; remote pump control; calibration.

PRELIMINARY PAGES

Executive summary

The project addresses a familiar problem in plant care: the user may not know the current moisture condition of the soil and may not be physically present near the plant to operate a pump. A connected sensor and dashboard can make the condition visible and allow a user to act remotely. The prototype combines an analog sensing element, an ESP8266 Wi-Fi microcontroller, Blynk cloud communication and a relay-switched pump circuit.

What the current firmware does

- Samples the sensor connected to A0 through analogRead().
- Maps the raw reading from DRY_VALUE = 1023 and WET_VALUE = 450 into a 0–100 display range, then constrains the result.
- Prints the raw value and mapped percentage to the serial monitor.
- Publishes the moisture indicator to Blynk virtual pin V0 when the cloud connection is active.
- Receives a 0/1 command on virtual pin V1 and operates the active-LOW relay output at D4.
- Commands the pump OFF in setup() and after a Blynk connection event.

What the report does not claim

No measured water-saving percentage, sensor accuracy, pump flow rate, response latency or long-duration reliability value is claimed because a controlled data set was not supplied. Automatic moisture-threshold pump control, hysteresis, reservoir-level interlocking and offline fallback are identified as future work.

Implementation baseline

The final source code supplied by the team is treated as authoritative. Earlier planning references to D3, Blynk Legacy and a self-hosted Java server are not part of the current implementation; the final code uses D4 and blynk.cloud.

PRELIMINARY PAGES

Table of contents

Section	Coverage
Preliminary pages	Declaration, acknowledgement, abstract, executive summary and lists
Chapter 1 · Introduction	Context, motivation, problem statement, objectives and scope
Chapter 2 · Technical foundations	IoT, moisture sensing, ADC mapping, ESP8266 and Blynk IoT
Chapter 3 · Requirements and constraints	Functional behaviour, use cases and non-functional requirements
Chapter 4 · System architecture	Overall architecture, telemetry path and manual command flow
Chapter 5 · Hardware design	BOM, pin assignment, sensor, relay, power and wiring
Chapter 6 · Firmware design	Configuration, helper functions, callbacks, telemetry and main loop
Chapter 7 · Calibration	Mapping calculation and calibration experiment procedure
Chapter 8 · Assembly and testing	Assembly, commissioning, functional test matrix and evidence
Chapter 9 · Safety and reliability	Electrical safety, fail-safe behaviour and cybersecurity
Chapter 10 · Scalability and conclusion	Multi-node possibilities, future work and team contributions
References	Technical documentation and source notes
Appendix A · Firmware	Complete sanitized firmware listing
Appendix B · Worksheets	Calibration sheet, test record, glossary and handover checklist
Appendix C · Troubleshooting	Practical symptom and corrective-action matrix
Appendix D · Viva preparation	Frequently asked technical and evaluation questions
Appendices E–R · Extended technical record	Cloud provisioning, data contract, numerical calibration, sensor uncertainty, electrical and hydraulic design, commissioning, test records, metrics, maintenance, future automatic mode, team handover and extended viva

Reading the report

The report follows the final firmware supplied by the team. It distinguishes current monitoring and manual pump control from automatic threshold-based irrigation, which remains proposed future work.

PRELIMINARY PAGES

List of figures, tables and abbreviations

Figures

- Figure 1. Overall system architecture
- Figure 2. Firmware execution and bidirectional data flow
- Figure 3. Conceptual low-voltage DC wiring arrangement
- Figure 4. Code-derived moisture display mapping

Tables

- Table 1. Key firmware constants and pin assignments
- Table 2. Hardware bill of materials and verification notes
- Table 3. Functional requirements
- Table 4. Non-functional requirements
- Table 5. Virtual-pin data contract
- Table 6. Functional test matrix
- Table 7. Risk assessment
- Table 8. Team responsibility matrix

CHAPTER 1 · INTRODUCTION

1.1 Introduction and context

Plant watering is a recurring task in homes, laboratories, nurseries and agricultural environments. A simple watering routine may rely on visual inspection, a fixed timer or manual operation of a pump. These approaches can be adequate for small setups, but they offer limited information about the moisture condition below the soil surface and may be inconvenient when the user is away.

The Internet of Things provides a way to connect physical sensors and actuators to software services. A microcontroller can collect an analog measurement, perform a calculation, send the result over Wi-Fi and expose it through a mobile or web dashboard. A command can then travel from the dashboard to the device, where the microcontroller changes an output pin.

The project applies this pattern to a single plant-watering point. A soil-moisture sensor is connected to the analog input of a NodeMCU ESP8266. The firmware converts the raw reading into a percentage-like display value and sends it to Blynk. A separate dashboard switch sends a manual ON/OFF command to a relay output that controls the pump circuit.

The title “Automatic Plant Irrigation System” expresses the application area and intended direction of development. The current firmware, however, automates sensing and telemetry rather than the final irrigation decision. This report uses precise wording so that the project is not represented as having threshold-based automatic pump control before that logic is implemented and tested.

CHAPTER 1 · INTRODUCTION

1.2 Background and motivation

Soil moisture is an important condition for plant growth, but the moisture surrounding a root system is not always obvious from the soil surface. Sensor-based monitoring provides a repeatable electrical signal that can be observed over time. Even when the displayed number is only a relative indicator, a trend can help the user compare wet and dry conditions in the same installation.

A connected design also separates measurement from the user interface. The sensor and microcontroller remain near the plant, while the dashboard can be viewed on a phone or browser. The user does not have to stand beside the plant to see the current reading or send a pump command, provided the device has Wi-Fi and cloud connectivity.

The project is motivated by the educational value of integrating mechanical application needs with electronics and software. The pump and water path form the physical process; the sensor provides feedback; the ESP8266 performs local computation; and Blynk provides a cloud-connected user interface. The system is small enough to understand component by component while illustrating the structure of a larger IoT solution.

Engineering motivation

The goal is not to claim a universal irrigation recipe. It is to create a connected prototype in which the sensor state is visible and the pump can be operated remotely, then provide a sound basis for calibration and future automation.

CHAPTER 1 · INTRODUCTION

1.3 Problem statement

The practical problem addressed is the limited availability of timely soil-moisture information and the inconvenience of operating a watering pump locally. A fixed timer does not respond to the actual soil condition. Manual inspection provides only intermittent observations and may be difficult when a plant is located away from the user.

Problem definition

Problem statement

Design and document a low-cost IoT prototype that measures soil moisture, makes the measured condition visible on a cloud dashboard and allows the user to operate a DC water pump remotely through a relay-controlled output.

Engineering questions

- How can a soil-moisture sensor be interfaced safely with the ESP8266 analog input?
- How can the raw ADC reading be converted into a useful bounded indicator?
- How can telemetry be sent periodically without flooding the cloud service?
- How can a dashboard command be translated into a physical relay output?
- How should the pump load be powered and isolated from the controller logic?
- What must be tested before the prototype is suitable for unattended operation?

The project scope is a single sensing and actuation node. It does not attempt to solve farm-scale irrigation scheduling, crop water requirements or hydraulic distribution design.

CHAPTER 1 · INTRODUCTION

1.4 Objectives, scope and assumptions

Objectives

- Acquire an analog soil-moisture reading through NodeMCU pin A0.
- Convert the raw ADC reading into a 0–100 moisture display indicator using selected dry and wet endpoints.
- Publish the indicator to Blynk virtual pin V0 at a one-second interval.
- Receive a remote manual pump command through Blynk virtual pin V1.
- Control an active-LOW relay using NodeMCU pin D4.
- Ensure the firmware commands the pump OFF during startup and after a Blynk connection event.
- Document hardware safety, calibration, testing and future automation requirements.

Scope boundaries

The report covers one ESP8266 node, one analog moisture input, one relay output, one DC pump interface and one Blynk cloud device/dashboard. It includes firmware explanation, a reference wiring concept, calibration procedure, test plan, risk analysis and possible extensions.

It excludes verified field trials, measured water savings, crop-specific irrigation thresholds, multi-zone deployment and fully automatic threshold-based pump operation. Hardware details that depend on the exact sensor and relay module are marked for verification rather than assumed.

Assumptions to verify

The NodeMCU board variant, A0 input range, sensor module supply voltage, relay input compatibility, relay contact rating, pump rated voltage/current and physical COM/NO wiring must be checked against the actual components before power is applied.

CHAPTER 2 · TECHNICAL FOUNDATIONS

2.1 IoT system fundamentals

An IoT system links physical objects to digital services through a sequence of sensing, processing, communication and action. In this project, the sensor produces an electrical output related to soil condition. The microcontroller samples and transforms the signal. Wi-Fi provides network connectivity. Blynk stores or routes device data for display and delivers a user command back to the device.

Functional layers

Layer	Project element	Responsibility
Physical sensing	Soil-moisture module	Produces analog signal representing local soil condition

This layered view helps with troubleshooting. A sensor fault is different from a Wi-Fi failure; a cloud authentication issue is different from a relay wiring problem. Each layer should be verified independently before the complete system is tested.

CHAPTER 2 · TECHNICAL FOUNDATIONS

2.2 Soil-moisture sensing principle

An analog soil-moisture module produces a voltage that changes with the electrical properties of the material around its probe. Many low-cost modules infer moisture from the change in conductivity or impedance of the soil-probe system. As the water content changes, the electrical path changes, and the module conditions the signal for an analog output.

The relationship between soil moisture and ADC value depends on the actual probe construction and signal-conditioning circuit. A common resistive probe gives a lower raw reading in wetter conditions and a higher raw reading in drier conditions, but this direction must be verified on the exact module used by the team. The firmware assumes that the wet endpoint has a lower reading than the dry endpoint.

Installation considerations

- Insert the probe at a repeatable depth in the root zone.
- Avoid placing the probe directly beside the water outlet, pot wall or a localized drainage channel.
- Keep module electronics and exposed connections away from splashing water.
- Use stable wiring and avoid moving the probe between calibration and routine measurements.
- Inspect the sensing surface for corrosion, deposits and mechanical damage.

Sensor readings should be interpreted as local measurements, not as a complete representation of the moisture distribution in a large pot or field. Site-specific calibration and representative placement are essential [6].

CHAPTER 2 · TECHNICAL FOUNDATIONS

2.3 ADC values, calibration and moisture indicators

The ESP8266 includes a 10-bit successive-approximation ADC, producing a digital reading in the nominal range 0 to 1023 [4]. The exposed A0 input range depends on the development-board circuit. The ESP8266EX chip-level TOUT input range is specified as 0–1.0 V; many NodeMCU-style boards add a divider, but the exact board schematic must be checked before applying a signal.

The firmware sets DRY_VALUE to 1023 and WET_VALUE to 450. These are software calibration endpoints. The mapping converts a raw reading into a relative indicator between zero and one hundred. The resulting number is useful for observing trends within the calibrated installation, but it is not automatically a true volumetric water-content percentage.

Why calibration matters

Soil texture, bulk density, salinity, temperature, probe placement, sensor ageing and supply variation can influence the reading. A published sensor guide recommends site-specific calibration and periodic validation against representative soil conditions [6]. A generic endpoint pair should therefore be treated as provisional until checked in the actual potting soil.

Terminology

In this report, “moisture percentage” means the firmware’s normalized 0–100 display indicator. The report does not label it as measured VWC unless a separate soil-specific calibration establishes that relationship.

CHAPTER 2 · TECHNICAL FOUNDATIONS

2.4 NodeMCU ESP8266 controller

The NodeMCU ESP8266 development board combines a microcontroller, Wi-Fi radio, flash storage and convenient board-level connections. It executes the Arduino-style C++ sketch, samples A0, runs the Blynk library callbacks, schedules the sensor routine and drives the relay input.

Pins used by the submitted firmware

Board label	Role in project	Notes
A0	Analog sensor input	Check the A0 voltage limit of the exact NodeMCU board.
D4	Relay input	Maps to GPIO2 on common NodeMCU mappings [5].
3V3 / VIN	Board supply options	Use the input specified by the particular board; do not assume VIN and 3V3 are interchangeable.
GND	Common reference	Logic-side ground wiring must follow the selected relay module design.

The ESP8266 is a 3.3 V logic device. External modules must be compatible with its logic levels. D4 corresponds to GPIO2 in the ESP8266 Arduino board mapping [5], and GPIO2 participates in boot-mode selection [4]. The relay interface must not force an invalid level during reset. Boot behaviour should be tested with the pump disconnected.

CHAPTER 2 · TECHNICAL FOUNDATIONS

2.5 Blynk IoT Cloud and virtual pins

Blynk provides a cloud-connected device and dashboard environment. The firmware identifies the device through a template configuration and an authentication token, joins the configured Wi-Fi network and connects to the Blynk Cloud endpoint. The current sketch explicitly supplies the host “blynk.cloud” and port 80 in Blynk.begin().

Virtual pins are logical data channels. They are not physical GPIO pins. Blynk documents Blynk.virtualWrite() for sending data from hardware and BLYNK_WRITE() for receiving application or server updates [1]. The distinction is central to this design: V0 and V1 are software channels, while A0 and D4 are physical NodeMCU pins.

Virtual pin	Direction	Value in this project
V0	Device → cloud/dashboard	Integer moisture indicator, 0–100
V1	Dashboard/cloud → device	Manual pump command, 0 = OFF and 1 = ON

The BlynkTimer is used to schedule periodic telemetry instead of calling Blynk.virtualWrite() continuously inside loop(). Blynk recommends timed sending to avoid excessive messages and possible disconnection [2]. Blynk.run() should be called frequently to process communication and connection housekeeping [3].

CHAPTER 3 · REQUIREMENTS AND CONSTRAINTS

3.1 System requirements

Requirements translate the project objective into verifiable functions and constraints. The requirements below describe the submitted firmware and the hardware interface that must be verified during assembly.

ID	Requirement	Verification evidence
FR-01	Read analog soil sensor on A0.	Serial Monitor raw ADC values change with probe condition.

ID	Requirement	Verification evidence
FR-02	Convert raw ADC value to a 0–100 indicator.	Check endpoint and intermediate calculations.
FR-03	Publish moisture indicator on V0.	Dashboard V0 value updates while cloud connected.
FR-04	Receive manual pump command on V1.	V1=1 and V1=0 invoke corresponding callback branches.
FR-05	Switch relay through D4 using active-LOW logic.	Verify relay state with pump disconnected.
FR-06	Command pump OFF at startup and after connection.	Observe relay output during boot and Blynk reconnect.

The requirements do not include automatic pump switching based on a moisture threshold because no such comparison is present in the latest submitted code. This boundary is important when evaluating functionality.

CHAPTER 3 · REQUIREMENTS AND CONSTRAINTS

3.2 Functional behaviour and use cases

Primary use cases

1. View moisture: the user opens the Blynk dashboard and reads the current V0 indicator.
2. Operate pump: the user turns the V1 switch ON or OFF. Blynk sends the value to the device, and BLYNK_WRITE(V1) invokes pumpOn() or pumpOff().
3. Reconnect device: when the device establishes a Blynk connection, BLYNK_CONNECTED() commands pump OFF and writes zero to V1.
4. Inspect sensor locally: the user opens the Serial Monitor at 9600 baud and observes raw ADC value and mapped percentage.

State interpretation

Event	Firmware action	Expected physical state
Power-up	pinMode(D4, OUTPUT); pumpOff()	Relay commanded OFF
Blynk connects	pumpOff(); virtualWrite(V1,0)	Relay OFF; dashboard command reset to 0
V1 receives 1	pumpOn()	D4 LOW; relay energized
V1 receives 0	pumpOff()	D4 HIGH; relay de-energized
Timer tick	Read A0; calculate; write V0	No direct pump change

Key distinction

The sensor routine reports a value; it does not call pumpOn() or pumpOff(). The current watering decision is made by the user, not by a moisture threshold.

CHAPTER 3 · REQUIREMENTS AND CONSTRAINTS

3.3 Non-functional requirements and constraints

Category	Requirement / design target	Current status
Safety	Pump power must not be drawn from a GPIO pin. Use a suitable relay and external DC source.	Design requirement; verify wiring.
Fail-safe	Pump command should default to OFF at boot and after Blynk connection.	Implemented in code; physical output must be tested.
Responsiveness	Blynk.run() must be serviced frequently; avoid blocking delays.	Main loop follows this pattern.
Data rate	Send sensor values periodically, not continuously.	BlynkTimer interval = 1000 ms.

Category	Requirement / design target	Current status
Accuracy	Moisture display must be calibrated for the installed sensor and soil.	Endpoints are provisional.
Availability	Remote control depends on Wi-Fi and Blynk Cloud.	No offline fallback in current code.
Security	Keep auth token and Wi-Fi password private.	Credentials redacted from this report.

The design is a prototype, not a certified irrigation controller. Reliability targets such as uptime, measurement error, pump runtime and response delay must be defined and measured before deployment outside a supervised demonstration environment.

CHAPTER 4 · SYSTEM ARCHITECTURE

4.1 Overall system architecture

The architecture is divided into a sensor path, an edge-processing path, a cloud telemetry path and a manual actuation path. The sensor provides an analog signal to A0. The ESP8266 calculates the display indicator and publishes it to V0. A dashboard switch sends a command through V1. The device callback changes D4, which drives the relay input.

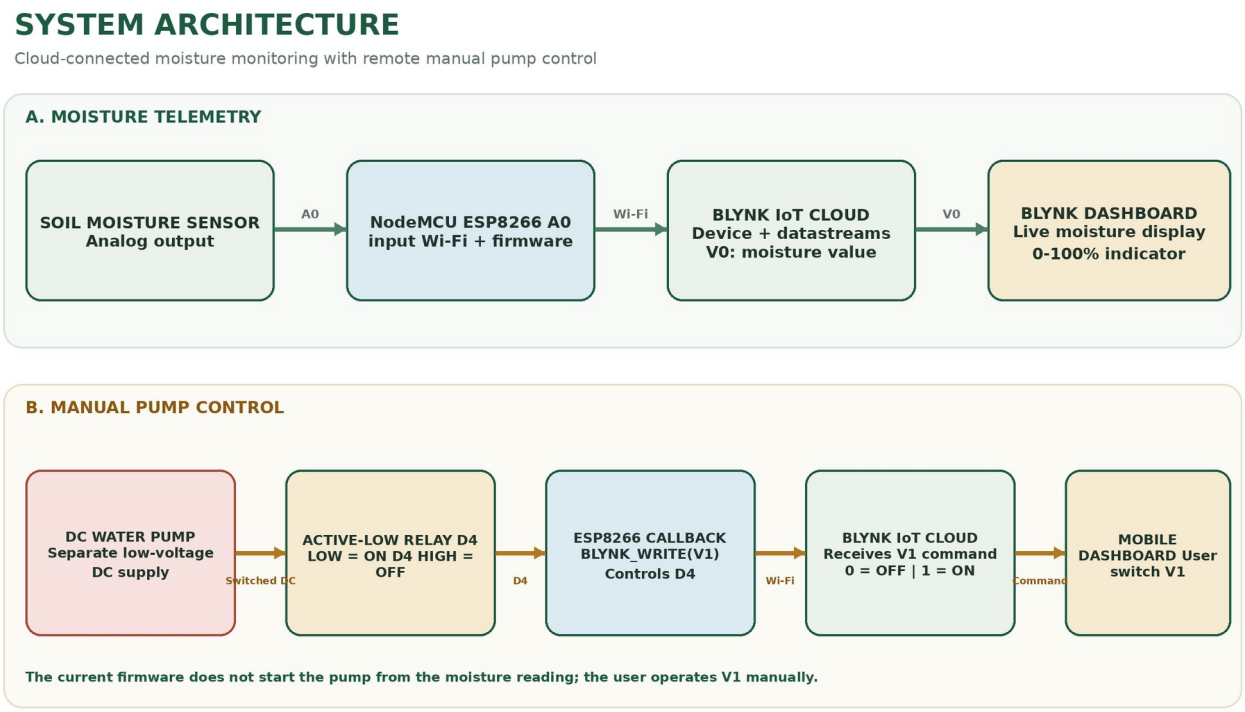


Figure 1. Overall system architecture. The upper path publishes sensor telemetry; the return path carries a manual dashboard command.

The pump is switched through relay contacts and uses a separate low-voltage DC supply matched to its rated voltage and current. The NodeMCU output controls only the relay input circuit; it must not carry motor current. The diagram is conceptual and must be reconciled with the exact component labels before wiring.

CHAPTER 4 · SYSTEM ARCHITECTURE

4.2 Firmware and communication flow

Two different events drive the firmware. A timer event performs measurement and telemetry. A cloud callback performs manual pump actuation. A third event, BLYNK_CONNECTED(), imposes a safe OFF command after connection.

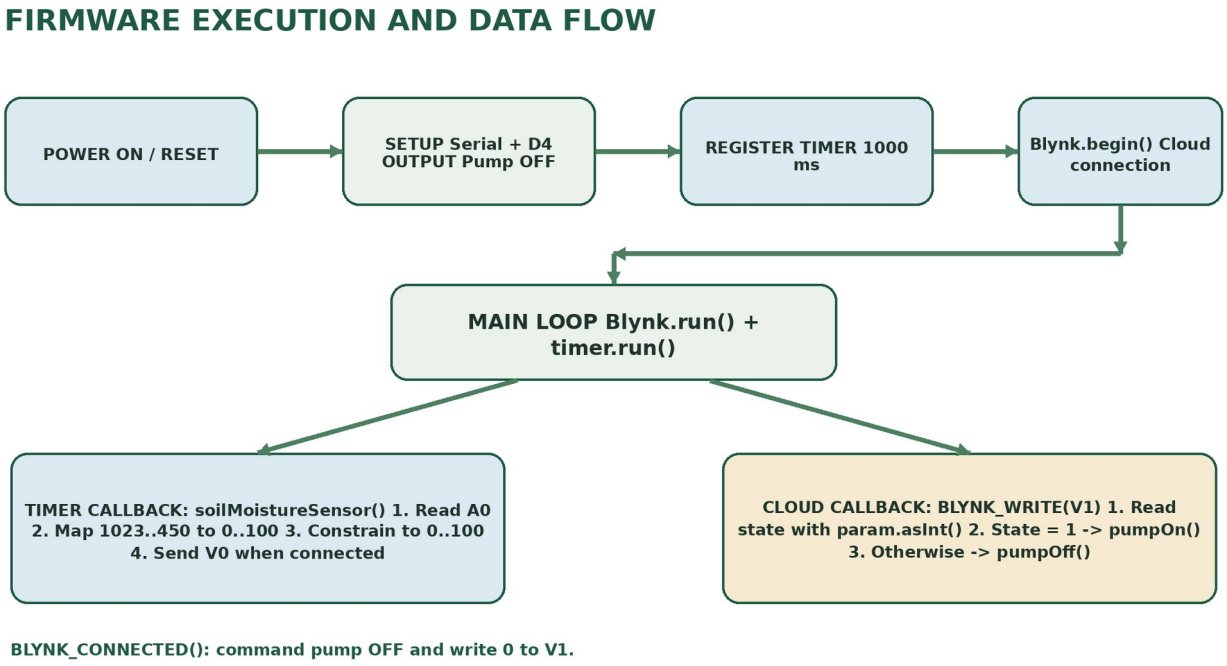


Figure 2. Firmware execution and data flow. The sensor timer does not command the pump; pump commands are handled by V1.

Separating these pathways prevents a common misunderstanding: a system can monitor moisture automatically without automatically deciding when to irrigate. In the submitted firmware, the sensor routine and pump-control callback are separate functions. Any future automatic mode must add an explicit policy that links measured moisture to pump state.

CHAPTER 5 · HARDWARE DESIGN

5.1 Bill of materials and component verification

The following bill of materials describes the functional elements required by the submitted firmware and system concept. Exact manufacturers, model numbers and electrical ratings were not supplied, so they must be filled in from the physical parts used by the team.

Item	Purpose	Specification to verify
NodeMCU ESP8266 board	Sampling, processing and Wi-Fi	Board revision; A0 input range; USB/regulator rating
Analog moisture module	Soil condition measurement	Sensor technology; VCC range; analog output range
1-channel relay module	Switch pump power path	Input trigger polarity; logic compatibility; DC contact rating
DC water pump	Water delivery	Rated voltage; running current; startup/stall current; flow rate
External DC supply	Pump power	Voltage and current capacity; polarity; protection

Item	Purpose	Specification to verify
Breadboard / wires / tubing	Prototype assembly	Current suitability; insulation; tube diameter and leakage

Procurement and documentation note

Do not infer component ratings from appearance. Record the exact module markings or datasheet values before making the final wiring diagram.

CHAPTER 5 · HARDWARE DESIGN

5.2 NodeMCU pin assignment and electrical limits

The latest firmware uses the board labels A0 and D4. On common NodeMCU ESP8266 mappings, D4 corresponds to GPIO2 [5]. The ESP8266 datasheet identifies GPIO2 as one of the pins used for boot-mode selection [4]. It is therefore important that the attached relay interface does not pull the pin to an invalid level during reset.

Pin assignment table

Signal	Board pin	Direction	Purpose
Sensor output	A0	Input	Analog moisture measurement
Relay input	D4	Output	Active-LOW relay trigger
Serial debug	TX/RX over USB serial	Bidirectional	9600 baud diagnostic output
Logic ground	GND	Reference	Common logic reference as required by module wiring

Board-specific precautions

The ESP8266EX chip-level ADC input is specified for 0–1.0 V [4], while development boards may incorporate a voltage divider on the exposed A0 pin. Never assume the same A0 range across boards. Confirm the board schematic or manufacturer specification. Similarly, verify D4 boot behaviour with the selected relay module and begin relay testing with the pump disconnected.

CHAPTER 5 · HARDWARE DESIGN

5.3 Moisture sensor interfacing and installation

The sensor module should be powered according to its own specification. Its analog output is connected to A0. The module ground and controller reference must be connected as required by the selected module topology. The output must remain within the A0 input range of the actual NodeMCU board.

Installation procedure

5. Identify sensor VCC, GND and AO terminals from the module markings or datasheet.
6. Check voltage compatibility before connecting VCC to the controller.
7. Connect AO to A0 and secure the signal and ground wires.
8. Place the probe at a representative depth in the root zone.
9. Record the raw reading in dry and wet reference conditions.
10. Repeat measurements and verify that the displayed percentage moves in the expected direction.

A resistive probe may corrode over time; a capacitive probe may reduce electrode corrosion but still requires installation-specific calibration. The actual sensor type in this project was not confirmed by a part number. Sensor maintenance and calibration are recommended for reliable measurements [6].

CHAPTER 5 · HARDWARE DESIGN

5.4 Relay module and active-LOW pump control

A relay is an electrically operated switch. The NodeMCU drives the relay module input; the relay contacts switch the separate pump power path. This prevents the motor current from being supplied directly by the microcontroller output pin.

Firmware function	D4 output	Relay state	Commanded pump state
pumpOff()	HIGH	De-energized	OFF
pumpOn()	LOW	Energized	ON

The code assumes an active-LOW relay module. Actual relay boards vary, so confirm the input logic before connecting the pump. With the pump connected through COM and NO, the circuit should be open while the relay is de-energized and closed when energized. Verify terminal markings and the DC contact rating, including motor startup current.

Relay verification without the pump

- Power the controller and relay module using compatible supplies.
- Observe the relay indicator or listen for the switching action when V1 changes.
- Measure contact continuity with the pump disconnected.
- Confirm that HIGH means OFF and LOW means ON for the actual module.
- Only proceed to pump testing after the logic and contact path are verified.

GPIO limitation

The D4 GPIO signal is a control signal only. Never connect the pump directly to D4 or any other ESP8266 GPIO pin.

CHAPTER 5 · HARDWARE DESIGN

5.5 Power supply, isolation and load protection

The NodeMCU and sensor require a regulated low-voltage logic supply appropriate to the board and sensor. The pump requires a separate DC supply selected from its nameplate or datasheet. Relay contacts switch the pump supply. This arrangement keeps motor current out of the NodeMCU regulator and signal wiring.

Electrical design checks

- Confirm pump voltage, rated current and startup or stall current.
- Select a relay with a DC contact rating suitable for the motor load.
- Use a supply with adequate current capacity and appropriate overcurrent protection.
- Use correct polarity, insulated terminals and wire gauge suitable for the expected current.
- Keep the water reservoir, pump and tubing physically separated from exposed electronics.
- Add suppression components only in accordance with the pump/relay manufacturer and the actual circuit design.

Optocouplers on relay modules can provide signal isolation only when the module layout and supply wiring preserve that isolation. Do not assume the presence of an optocoupler guarantees a fully isolated system. Avoid exposed mains wiring in a student breadboard prototype. This report uses a low-voltage DC pump design.

CHAPTER 5 · HARDWARE DESIGN

5.6 Reference wiring arrangement

The diagram below is a low-voltage DC conceptual arrangement based on the final code pin assignments. It is not a substitute for the exact schematic of the purchased modules. The logic wiring and the load wiring are shown separately to emphasize that the pump is powered by an external DC source.

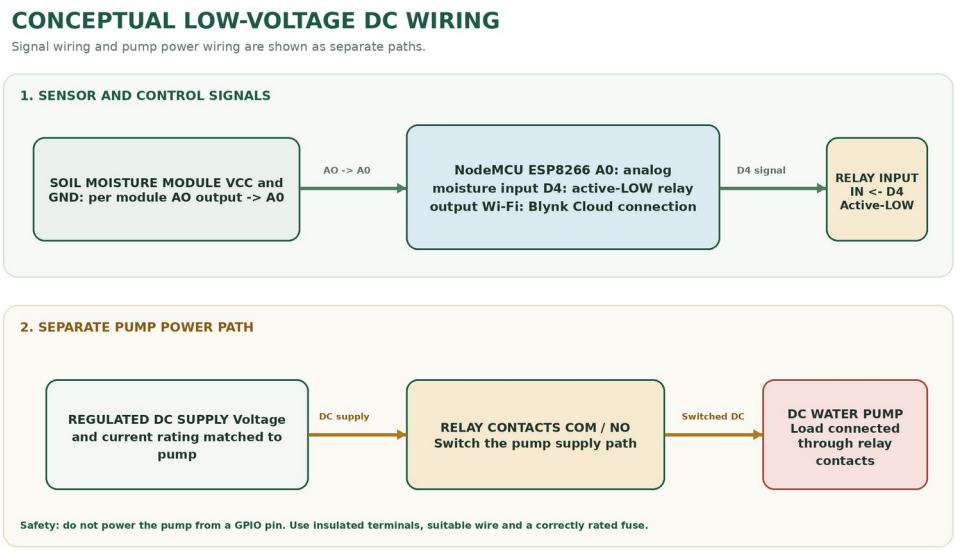


Figure 3. Conceptual low-voltage DC wiring. Verify the actual board A0 range, relay trigger polarity, COM/NO labels and pump supply ratings before assembly.

Mandatory pre-power checklist

Confirm A0 voltage compatibility; confirm D4 active-LOW behaviour; verify relay contact ratings; check DC supply polarity; inspect for exposed conductors; keep water away from electronics; test first with the pump disconnected.

CHAPTER 6 · FIRMWARE DESIGN

6.1 Firmware architecture

The sketch is organized around global configuration values, two pump helper functions, two Blynk callbacks, one sensor routine, a setup function and a main loop. This structure separates the hardware actions from the cloud event handling and from the periodic measurement task.

Firmware block	Role
Declarations	Define Blynk template/device metadata, credentials, libraries, timer, pins and calibration endpoints.
pumpOff()	Set D4 HIGH, record Relay=false and print the OFF message.
pumpOn()	Set D4 LOW, record Relay=true and print the ON message.
BLYNK_CONNECTED()	Command OFF and write 0 to V1 after connection.
BLYNK_WRITE(V1)	Read dashboard value and call pumpOn() or pumpOff().
soilMoistureSensor()	Read A0, map and constrain the reading, print diagnostics and send V0.
setup() / loop()	Initialize hardware and cloud; service Blynk and timer continuously.

The main loop is intentionally short. Blynk.run() processes network activity, while timer.run() checks whether the periodic sensor routine is due. Blynk recommends frequent servicing of Blynk.run() and timed scheduling of cloud writes [2, 3].

CHAPTER 6 · FIRMWARE DESIGN

6.2 Firmware configuration and credential handling

The program begins by defining the Blynk template ID and template name before including the ESP8266 Wi-Fi and Blynk libraries. It then places the authentication token in a named array and declares Wi-Fi SSID and

password variables. The actual source supplied to this report contained working credentials; they are deliberately replaced with placeholders here.

```
#define BLYNK_TEMPLATE_ID   "TMPL3QABczm15"
#define BLYNK_TEMPLATE_NAME "Smart Plant Watering System"
#define BLYNK_AUTH_TOKEN    "YOUR_BLYNK_AUTH_TOKEN"
#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "YOUR_WIFI_SSID";
char pass[] = "YOUR_WIFI_PASSWORD";
```

The token identifies and authorizes the device connection. The Wi-Fi credentials allow the board to join the local wireless network. Both values must be stored privately and must not appear in a public report, shared slide deck, screenshot or source repository. A token that has been exposed should be revoked or regenerated.

The Blynk documentation describes the template ID and device name as part of the firmware preparation process for supported ESP8266 boards [1]. The team should keep the template, device and firmware configuration consistent.

CHAPTER 6 · FIRMWARE DESIGN

6.3 Pump helper functions and relay state

```
void pumpOff() {
  digitalWrite(waterPump, HIGH);
  Relay = false;
  Serial.println("Motor is OFF");
}
void pumpOn() {
  digitalWrite(waterPump, LOW);
  Relay = true;
  Serial.println("Motor is ON");
}
```

The helper functions centralize the relay polarity. pumpOff() writes HIGH to D4. Because the selected relay logic is active-LOW, this de-energizes the relay. pumpOn() writes LOW, energizing the relay and closing the intended switched contact path. The Boolean Relay variable stores the commanded state for readability and future development; the current sketch does not use it to make later decisions.

Why helper functions are useful

- They avoid duplicating the active-LOW logic in multiple callback branches.
- They keep the serial debug message consistent with the output command.
- They make it easier to change relay polarity in one place if the hardware differs.
- They support isolated testing of ON and OFF behavior before the pump is connected.

Important distinction

The serial messages describe the firmware command. They do not independently prove that the pump physically received power or delivered water; that requires a physical test.

CHAPTER 6 · FIRMWARE DESIGN

6.4 Blynk callbacks and manual control

```
BLYNK_CONNECTED() {
  pumpOff();
  Blynk.virtualWrite(V1, 0);
}
BLYNK_WRITE(V1) {
  int state = param.asInt();
  if (state == 1) {
```

```
    pumpOn();
  } else {
    pumpOff();
  }
}
```

BLYNK_CONNECTED() is invoked when the device connects to Blynk Cloud. The project uses this event to command the pump OFF and publish zero to V1. This is intended to establish a predictable starting state after a successful connection. The physical relay output must still be checked during boot and reconnection.

BLYNK_WRITE(V1) is the event handler for a value received on virtual pin V1. The statement param.asInt() converts the incoming virtual-pin value into an integer. A value of 1 invokes pumpOn(); any other value invokes pumpOff(). Blynk virtual pins are logical channels rather than physical GPIO pins [1].

The current code uses V1 for manual actuation. It does not read V0 inside this callback and does not apply a moisture threshold. A future automatic mode should be added as a separate, well-tested control policy rather than inferred from the existing callback.

6.5 Sensor acquisition, mapping and V0 telemetry

```
void soilMoistureSensor() {
  int raw = analogRead(sensor);
  int moisture = map(raw, DRY_VALUE, WET_VALUE, 0, 100);
  moisture = constrain(moisture, 0, 100);
  Serial.print("Raw: ");
  Serial.print(raw);
  Serial.print("  Moisture: ");
  Serial.print(moisture);
  Serial.println("%");
  if (Blynk.connected()) {
    Blynk.virtualWrite(V0, moisture);
  }
}
```

analogRead(sensor) samples the A0 input and stores the result in raw. The map() function scales the reading from the dry-to-wet endpoint range into the output range 0–100. constrain() clips the mapped value to prevent values below zero or above one hundred from being displayed.

The Serial Monitor output provides two useful diagnostic values: the raw ADC reading and the mapped indicator. If Blynk.connected() returns true, Blynk.virtualWrite(V0, moisture) sends the indicator to the dashboard. The routine does not operate the relay. This separation is deliberate in the submitted version.

6.6 Timer scheduling, setup and main loop

```
void setup() {
  Serial.begin(9600);
  pinMode(waterPump, OUTPUT);
  pumpOff();
  timer.setInterval(1000L, soilMoistureSensor);
  Serial.println("Starting Smart Plant Watering System...");
  Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
}
void loop() {
  Blynk.run();
  timer.run();
}
```

setup() runs once after startup. It opens serial communication at 9600 baud, configures D4 as an output, commands the pump OFF, registers the sensor routine at a 1000-millisecond interval and starts the Blynk connection using the device token and Wi-Fi credentials.

loop() runs continuously. Blynk.run() processes network events and callbacks. timer.run() checks the timer schedule and calls soilMoistureSensor() when due. The Blynk documentation recommends using BlynkTimer for periodic writes and avoiding long blocking delays that interfere with communication [2, 3].

Runtime behaviour
The sensor routine is scheduled every 1 second. The dashboard update is sent only when Blynk.connected() is true. A lost Internet connection can prevent remote monitoring and control because this firmware has no local offline fallback.

CHAPTER 7 · CALIBRATION

7.1 Moisture display calculation

The submitted code uses Arduino map() with the input interval reversed: 1023 corresponds to the dry endpoint and 450 corresponds to the wet endpoint. The output interval is 0 to 100. The equivalent linear expression is:

Moisture indicator (%) = ((DRY_VALUE − ADC_raw) / (DRY_VALUE − WET_VALUE)) × 100

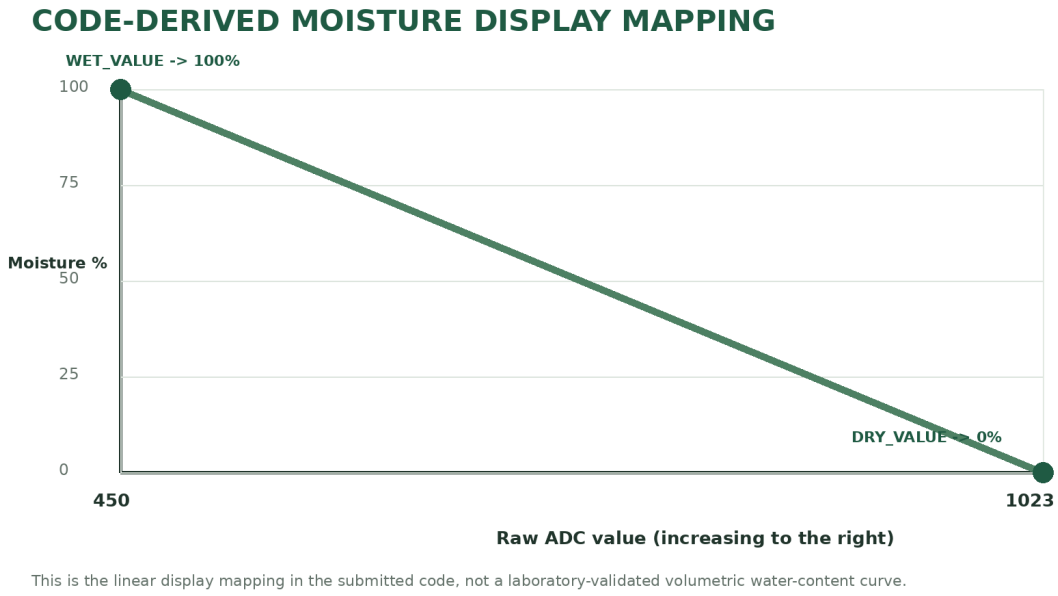


Figure 4. Linear mapping implied by DRY_VALUE = 1023 and WET_VALUE = 450.

Raw ADC input	Code-derived indicator	Interpretation under current assumptions
1023	0%	Dry calibration endpoint
736 or 737	Approximately 50%	Midpoint of the selected raw range
450	100%	Wet calibration endpoint

The expression is a normalized indicator. It should not be reported as true VWC without a soil-specific calibration reference. The mapping assumes a monotonic sensor response from the selected dry and wet endpoints. If measured readings reverse direction or vary substantially, the constants and possibly the mapping model must be revised.

CHAPTER 7 · CALIBRATION

7.2 Calibration experiment and data collection

Calibration should be performed on the actual sensor, supply, soil medium and installation depth. A robust procedure collects several readings at each reference condition and uses a representative statistic, such as the median or mean, rather than selecting one transient sample.

Recommended procedure

11. Record the sensor model, board variant, sensor supply voltage, soil type and probe depth.
12. Collect multiple raw ADC readings in the selected dry reference condition; record the full spread.
13. Place the probe in a consistently saturated reference sample without immersing electronics; allow the reading to stabilize.
14. Collect multiple raw readings in the wet reference condition.
15. Check that the wet/dry reading direction matches the firmware assumption.
16. Test intermediate moisture states in the target soil and compare with manual observations or an independent reference method.
17. Update DRY_VALUE and WET_VALUE only after documenting the measurements and repeating the check.

Soil-moisture sensor performance is affected by soil texture, salinity, temperature, probe placement and drift. A recent University of Arizona guide emphasizes site-specific calibration and periodic validation [6].

No fabricated results

The values 1023 and 450 are code constants supplied by the team. They are not presented here as measured calibration results from the final soil installation.

CHAPTER 8 · ASSEMBLY AND COMMISSIONING

8.1 Assembly procedure

Assembly should proceed from controller-only checks to sensor testing, relay testing and finally pump integration. This sequence reduces the chance that an incorrect wiring connection will expose the microcontroller to the pump load or cause unexpected actuation.

Recommended assembly sequence

18. Inspect the NodeMCU, sensor module, relay module and pump for damaged pins, loose terminals or exposed conductors.
19. Upload the firmware with private credentials stored locally and verify the board starts without the relay connected.
20. Connect the sensor module and verify raw ADC readings in the Serial Monitor.
21. Connect the relay input to D4 and verify active-LOW switching with the pump disconnected.
22. Wire the DC pump through COM/NO contacts and a correctly rated external DC supply.
23. Secure the pump tubing and reservoir so that water cannot drip onto the controller or relay.
24. Power the integrated setup under supervision and test V1 ON/OFF, V0 telemetry and the OFF-on-reconnect behaviour.

The exact VCC and ground wiring depends on the selected sensor and relay boards. Follow their datasheets and maintain the separation between the controller logic path and the pump load path.

CHAPTER 8 · ASSEMBLY AND COMMISSIONING

8.2 Testing methodology

Testing should verify each layer independently before the complete system is operated. The test record should include the date, hardware identification, firmware version, test condition, expected result, observed result, pass/fail decision and evidence filename.

Test phases

Phase	Focus	Evidence
1. Board boot	Board powers up and D4 reaches safe OFF command.	Serial startup log; relay state observation.
2. Sensor input	A0 responds when the probe moves between dry and wet conditions.	Raw ADC readings and calibration notes.
3. Mapping	Display indicator moves in the intended direction and stays within 0–100.	Calculated examples and serial output.
4. Cloud telemetry	V0 updates at the scheduled interval when connected.	Device online status and dashboard screenshot.
5. Manual actuation	V1=1 turns relay ON; V1=0 turns relay OFF.	Relay observation with pump disconnected.
6. Load integration	Pump starts/stops as commanded without overheating or leakage.	Video, current measurement and inspection.
7. Fault response	Observe Wi-Fi loss, cloud reconnect and controller restart.	Fault log and safe-state review.

Tests must be physically performed before being marked as passed. Code inspection is evidence of intended logic, not proof of physical behaviour.

CHAPTER 8 · ASSEMBLY AND COMMISSIONING

8.3 Functional test matrix

Test ID	Action / condition	Expected response	Status
T-01	Power up the controller.	D4 is configured as output and pumpOff() commands OFF.	To be recorded
T-02	Move sensor from dry to wet reference condition.	Raw ADC changes consistently; displayed indicator rises under code assumptions.	To be recorded
T-03	Check V0 while Blynk is connected.	Moisture indicator updates approximately every 1000 ms.	To be recorded
T-04	Set V1 to 1.	BLYNK_WRITE(V1) invokes pumpOn(); D4 LOW.	To be recorded
T-05	Set V1 to 0.	BLYNK_WRITE(V1) invokes pumpOff(); D4 HIGH.	To be recorded
T-06	Reconnect Blynk.	BLYNK_CONNECTED() commands OFF and writes 0 to V1.	To be recorded
T-07	Disconnect Wi-Fi while pump command is ON.	Observe behaviour; current firmware has no explicit offline stop policy.	To be recorded
T-08	Connect pump load.	Pump operates within rated electrical and hydraulic conditions.	To be recorded

Status convention

“To be recorded” is intentional. The team has not supplied completed test logs for this report; replace each status only after collecting evidence.

CHAPTER 8 • ASSEMBLY AND COMMISSIONING

8.4 Results, evidence and implementation status

The current project record contains the final firmware source and the Blynk cloud configuration details supplied by the team. It establishes the intended data flow and control mapping. No raw calibration log, measured pump current, flow-rate result, response-latency measurement or controlled water-consumption comparison was supplied for inclusion in this report.

Subsystem	Evidence available in the project record	What remains to document
Firmware	Complete latest sketch provided by the team.	Compiler/upload log and firmware version.
Blynk Cloud	Template and device configuration information; current code targets blynk.cloud.	Actual device online screenshot and V0/V1 dashboard evidence.
Moisture sensor	A0 input and mapping constants defined.	Measured dry/wet sample sets and soil-specific validation.
Relay	D4 active-LOW logic defined in code.	Physical relay test, including boot behaviour.
Pump circuit	Pump-control functions and relay switching design documented.	Exact pump rating, supply rating, current and water-delivery test.

This report distinguishes software intent from experimental evidence. A project may have a complete firmware listing while still requiring physical verification of wiring, calibration and the pump load. The next submission update should add original photographs and raw measurement records.

CHAPTER 9 • SAFETY AND RELIABILITY

9.1 Safety and risk assessment

Hazard / failure	Possible consequence	Control / mitigation
Wrong A0 voltage	ADC damage or incorrect readings.	Verify board A0 input range and sensor output voltage.
Relay polarity mismatch	Pump operates unexpectedly.	Test active-LOW behaviour with pump disconnected.
D4 boot interaction	ESP8266 boot failure or relay glitch.	Verify GPIO2 boot level; use suitable interface circuit.
Underrated relay contacts	Contact heating or failure.	Check DC motor and startup current ratings.
Pump supply overload	Voltage sag, overheating or wiring damage.	Select rated DC supply and protection.
Water ingress	Short circuit or equipment damage.	Separate wet and electrical areas; use enclosure and drip loops.
Pump runs unattended	Overflow or dry-run damage.	Add runtime limit, reservoir sensing and supervised operation.
Network loss	Remote command unavailable.	Define safe offline policy and local override.

The current code commands OFF during setup and after Blynk connection, but it does not define a maximum ON duration or an explicit stop action on network loss while the pump is already running. These are important future safety improvements.

CHAPTER 9 · SAFETY AND RELIABILITY

9.2 Cybersecurity, privacy and maintainability

The device authentication token and Wi-Fi password are credentials. Anyone who obtains them may be able to use the token or access the network, depending on the surrounding configuration. The report, presentation, screenshots and public repositories must therefore use placeholders. The actual token and password belong only in the private firmware source used by the team.

Credential and access practices

- Generate or rotate the Blynk device token after any accidental disclosure.
- Use a strong Wi-Fi password and avoid publishing the SSID/password pair in shared files.
- Do not commit credentials to a public source repository.
- Limit dashboard access to intended users and remove access when no longer required.
- Keep a private record of device identity, template association and token rotation date.
- Use a separate IoT network where practical and keep firmware dependencies under review.

Cloud connectivity provides remote access but introduces a dependency on the network and third-party service. The final code connects to blynk.cloud; this is not a local-only architecture. The team should verify the transport and device-security settings appropriate to its Blynk configuration before any deployment on an untrusted network.

CHAPTER 10 · SCALABILITY AND FUTURE WORK

10.1 Scalability and proposed extensions

The single-node prototype can be extended to multiple plant zones by deploying additional sensor-controller nodes. Each node would need a unique device identity, a defined telemetry schema and a corresponding actuator strategy. A central dashboard could show the moisture indicator and pump status for each zone.

Potential communication approaches

Approach	Potential role	Trade-off / requirement
Wi-Fi + Blynk Cloud	Direct cloud connection for each ESP8266 node.	Requires suitable Wi-Fi coverage and device configuration.
ESP-NOW	Short-range communication between ESP-based nodes and a gateway.	Requires gateway design, addressing and message handling.
LoRaWAN	Long-range, low-data-rate sensing across distributed zones.	Requires compatible radios, gateways and network coverage.

Future development priorities

- Implement local moisture-threshold control with hysteresis.
- Add a maximum pump runtime and minimum rest interval.
- Add reservoir-level detection and sensor-fault detection.
- Add offline local control so safe operation does not depend entirely on cloud access.
- Add trend history and exportable measurement records.
- Evaluate solar or battery supply only after measuring the actual node power profile.

These extensions are proposals. None of ESP-NOW, LoRaWAN, multi-node management or automatic threshold control is implemented in the submitted sketch.

CHAPTER 10 · CONCLUSION

10.2 Conclusion and team contributions

The project demonstrates a connected plant-watering prototype that joins analog sensing, embedded processing, cloud telemetry and remote actuation. The NodeMCU reads the moisture sensor on A0, computes a bounded indicator using the configured endpoints and sends the value to Blynk V0. A user command on V1 is handled by the firmware and translated into an active-LOW relay output on D4. The program commands the pump OFF at startup and after a successful Blynk connection.

The current version should be described as an IoT-based soil-moisture monitoring system with remote manual pump control. Automatic irrigation based on a moisture threshold is an important future extension, but it is not present in the final code. The main next steps are physical wiring verification, sensor calibration in the target soil, safe pump integration and evidence-based functional testing.

Team responsibility matrix

Team member	Registration no.	Primary contribution area
Anuj Parihar	20247015	Case study, scalability analysis, report organization and technical documentation.
Aman Kharwar	20247010	Hardware interfacing, sensor/relay/pump planning and firmware work.
Mayank Gupta	20247027	Blynk platform configuration, cloud device setup and system integration support.
Sudarshan Jha	20247051	Problem analysis, infrastructure research and technical system planning.

Final conclusion

The engineering value of the prototype lies in a clear, testable IoT data path and remote command path. Completing calibration, electrical verification and recorded tests will turn the documented design into a fully validated demonstration.

REFERENCES

Technical references and source notes

[1] Blynk Inc. “Virtual Pins.” Blynk Documentation. https://docs.blynk.io/en/blynk-library-firmware-api/virtual-pins (accessed 11 November 2026).

[2] Blynk Inc. “Timers.” Blynk Documentation. https://docs.blynk.io/en/blynk-library-firmware-api/blynk-timer (accessed 11 November 2026).

[3] Blynk Inc. “Connection Management.” Blynk Documentation. https://docs.blynk.io/en/blynk-library-firmware-api/connection-management (accessed 11 November 2026).

[4] Espressif Systems. “ESP8266EX Datasheet,” Version 7.1. https://documentation.espressif.com/0a-esp8266ex_datasheet_en.html (accessed 11 November 2026).

[5] ESP8266 Arduino Core. “Boards Reference.” https://github.com/esp8266/Arduino/blob/master/doc/boards.rst (accessed 11 November 2026).

[6] Elshikha, D., Attalah, S., Thorp, K. R., Williams, C., Wang, D., Kisekka, I., Alshraah, S., Dodge, A., Dull, A., Quanquin, B., Barr, A., and Elsadek, E. A. (2026). “A Guide to Maintaining and Calibrating Field-Installed Soil and Plant Moisture Sensors.” University of Arizona Cooperative Extension, Publication AZ2193, March 2026. https://extension.arizona.edu/publication/guide-maintaining-and-calibrating-field-installed-soil-and-plant-moisture-sensors (accessed 11 November 2026).

[7] Project team. “PlantWatering_NodeMCU.ino.” Final firmware supplied for this report, 2026. Private credentials redacted.

[8] Project team. “Expanded IoT Smart Plant Watering System Report.” Earlier technical draft supplied as source material, 2026. Used for project history and implementation notes; superseded by the latest firmware where differences exist.

Source hierarchy

The latest code supplied by the team defines the implementation details. Official Blynk and Espressif documentation supports platform and microcontroller explanations. The soil-moisture calibration guide supports the recommendation for site-specific calibration. Values not measured by the team are not presented as experimental results.

APPENDIX A · FIRMWARE LISTING

A.1 Final firmware — configuration and control functions

The following listing is based on the latest code supplied by the team. The Blynk token, Wi-Fi SSID and Wi-Fi password are replaced with placeholders. Populate these values only in the private development copy. Do not add credentials to this report or a public repository.

```
/* Automatic Plant Irrigation System - Blynk Cloud */
#define BLYNK_TEMPLATE_ID   "TMPL3QABczm15"
#define BLYNK_TEMPLATE_NAME "Smart Plant Watering System"
#define BLYNK_AUTH_TOKEN    "YOUR_BLYNK_AUTH_TOKEN"
#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "YOUR_WIFI_SSID";
char pass[] = "YOUR_WIFI_PASSWORD";
BlynkTimer timer;
#define sensor   A0
#define waterPump D4
const int DRY_VALUE = 1023;
const int WET_VALUE = 450;
bool Relay = false;
// Active-LOW relay: HIGH = OFF, LOW = ON.
void pumpOff() {
  digitalWrite(waterPump, HIGH);
  Relay = false;
  Serial.println("Motor is OFF");
}
void pumpOn() {
  digitalWrite(waterPump, LOW);
  Relay = true;
  Serial.println("Motor is ON");
}
```

Code note: DRY_VALUE and WET_VALUE are the selected mapping endpoints, not universal sensor constants. Verify them with measured data from the installed probe and soil. The variable Relay records the commanded relay state.

APPENDIX A · FIRMWARE LISTING

A.2 Final firmware — callbacks, telemetry and main loop

```
BLYNK_CONNECTED() {
  // Establish a safe OFF command after cloud connection.
  pumpOff();
  Blynk.virtualWrite(V1, 0);
}
BLYNK_WRITE(V1) {
```

```
  int state = param.asInt();
  if (state == 1) {
    pumpOn();
  } else {
    pumpOff();
  }
}
void soilMoistureSensor() {
  int raw = analogRead(sensor);
  int moisture = map(raw, DRY_VALUE, WET_VALUE, 0, 100);
  moisture = constrain(moisture, 0, 100);
  Serial.print("Raw: ");
  Serial.print(raw);
  Serial.print("  Moisture: ");
  Serial.print(moisture);
  Serial.println("%");
  if (Blynk.connected()) {
    Blynk.virtualWrite(V0, moisture);
  }
}
void setup() {
  Serial.begin(9600);
  pinMode(waterPump, OUTPUT);
  pumpOff();
  timer.setInterval(1000L, soilMoistureSensor);
  Serial.println("Starting Smart Plant Watering System...");
  Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
}
void loop() {
  Blynk.run();
  timer.run();
}
```

Functional summary

A0 = moisture input; D4 = active-LOW relay; V0 = moisture telemetry; V1 = manual pump command. Moisture monitoring is automatic; pump control is manual. No threshold-based automatic pump decision is present.

APPENDIX B · WORKSHEETS AND HANDOVER

B.1 Calibration sheet, test record and glossary

Calibration worksheet

Parameter	Record
Date / time	To be recorded during the test.
Sensor model / module	To be recorded during the test.
NodeMCU board variant	To be recorded during the test.
Soil type / potting medium	To be recorded during the test.
Probe depth / location	To be recorded during the test.
Sensor supply voltage	To be recorded during the test.
Dry raw readings (multiple samples)	To be recorded during the test.
Selected DRY_VALUE	To be recorded during the test.
Wet raw readings (multiple samples)	To be recorded during the test.
Selected WET_VALUE	To be recorded during the test.

Parameter	Record
Intermediate soil observations	To be recorded during the test.

Glossary and abbreviations

Term	Meaning
ADC	Analog-to-Digital Converter
Blynk virtual pin	Logical data channel; it is not a physical GPIO pin.
DC	Direct Current
GPIO	General-Purpose Input/Output
IoT	Internet of Things
SSID	Service Set Identifier; the Wi-Fi network name.
Telemetry	Sensor data sent from the device to a service.
Active-LOW	The relay input is asserted when driven LOW.
Hysteresis	Use of separate start and stop thresholds to prevent rapid switching.
VWC	Volumetric Water Content; not directly established by the current two-point indicator.

APPENDIX B · WORKSHEETS AND HANDOVER

B.2 Final submission and handover checklist

Handover item	Evidence to attach or verify	Done
Firmware source	Sanitized listing and private credential handling.	Pending
Blynk configuration	Template/device view, V0 datastream and V1 switch.	Pending
Sensor wiring	Photograph showing AO connected to A0 and correct supply.	Pending
Relay wiring	D4 input; active-LOW state tested with pump disconnected.	Pending
Pump supply	DC voltage/current rating, polarity and protection.	Pending
Calibration	Measured dry/wet endpoints and actual soil observations.	Pending
Telemetry test	Serial log and V0 dashboard update evidence.	Pending
Manual control test	V1 ON/OFF and relay/pump observations.	Pending
Safety review	Water/electronics separation, fuse and enclosure checks.	Pending
Credential security	No live token or Wi-Fi password in shared artifacts.	Pending

The final report should be updated with original photographs of the team's assembled prototype, actual Blynk dashboard screenshots, measured calibration readings and completed test records. Concept images and reference photographs are not evidence that a specific hardware test has passed.

End of report

This document records the current cloud-connected moisture-monitoring and manual pump-control implementation, its engineering basis, its known limitations and the evidence required for final validation.

APPENDIX C · TROUBLESHOOTING

C.1 Practical troubleshooting matrix

Use the checks below in sequence; record the observed symptom before changing wiring or code.

Symptom	Possible causes	Checks / corrective action
Device does not appear online	Wi-Fi credentials incorrect; weak signal; invalid token; wrong template/device association.	Check Serial Monitor, SSID, password, token, template ID, internet access and Blynk device status.
V0 does not update	Timer callback not running; Blynk disconnected; datastream/widget mismatch.	Confirm timer.setInterval(1000L,...), Blynk.run(), Blynk.connected(), V0 datastream type and dashboard widget binding.
Moisture percentage moves in the wrong direction	Sensor polarity or module behavior differs from the assumed mapping.	Record raw ADC values in dry and wet conditions. Swap calibration endpoints only after verifying repeated measurements.
Display is stuck at 0% or 100%	Raw readings lie outside the configured endpoints or endpoints are unsuitable.	Print raw values, check sensor wiring and supply, then recalibrate DRY_VALUE and WET_VALUE.
V1 switch does not operate relay	Dashboard writes wrong virtual pin/type; callback not triggered; device offline.	Check V1 datastream and switch settings, confirm 0/1 values, inspect Serial output and BLYNK_WRITE(V1).
Relay operates in reverse	Relay board is active-HIGH or relay input logic differs from assumption.	Test with pump disconnected. Confirm the relay truth table and adjust pumpOn()/pumpOff() only after verification.
ESP8266 fails to boot reliably	D4 maps to GPIO2 on common NodeMCU boards; relay board may load the boot strap pin.	Disconnect relay input and reboot. Check board documentation and consider a different safe GPIO if required.
Pump does not run although relay clicks	Pump supply, polarity, current rating, fuse, contacts or wiring may be incorrect.	Measure the DC supply, verify COM/NO path and motor startup current, and test the pump separately using its rated supply.
Remote control stops during internet outage	The current design depends on Wi-Fi and Blynk Cloud for remote commands.	Restore network access. Treat offline automatic fallback as future work; do not assume the current sketch provides it.

Safe troubleshooting order

Start with USB/serial diagnostics and sensor readings. Verify relay behavior with the pump disconnected. Connect the pump only after confirming voltage, polarity, current rating, relay contact rating and insulation.

Diagnostic order

A disciplined troubleshooting process changes one variable at a time. Begin with the least invasive checks: inspect power and wiring, read the Serial Monitor, verify the raw ADC value, confirm the Blynk device state and then test the dashboard datastream. Only after the sensor and network paths are understood should the relay path be investigated. Disconnect the pump before changing relay wiring or changing the GPIO configuration.

Record the symptom exactly as observed, including whether it is continuous or intermittent. For example, “V0 remains at 0 while raw ADC changes” is more useful than “sensor not working.” The first observation points toward mapping, Blynk connectivity or datastream configuration; the second does not isolate the fault. After a correction, repeat the original test and one related regression test.

APPENDIX D · VIVA PREPARATION

D.1 Frequently asked technical questions

Concise answers based on the submitted firmware and documented design.

Question	Answer
What does the current project do?	It monitors soil moisture and sends a mapped percentage to Blynk V0. A user controls the pump manually through V1.
Is pump operation automatic?	No. Sensor acquisition and telemetry are automatic, but the firmware does not compare moisture against a threshold to start or stop the pump.
Why use NodeMCU ESP8266?	It combines a microcontroller with Wi-Fi connectivity, allowing analog sensing, local processing and Blynk communication in one board.
What is A0 used for?	A0 receives the analog output of the soil-moisture sensor. The ESP8266 ADC converts the input into a digital reading.
What are DRY_VALUE and WET_VALUE?	They are mapping endpoints: 1023 is assigned to 0% and 450 to 100% in the current code. They must be verified with the actual probe and soil.
Does the display show true VWC?	Not necessarily. It is a normalized percentage-like indicator from a two-point linear mapping. True VWC requires soil-specific calibration against a reference method.
What is V0?	V0 is the Blynk virtual pin used for device-to-cloud moisture telemetry. It is not a physical GPIO pin.
What is V1?	V1 is the Blynk virtual pin used for the dashboard manual pump command. The callback reads its value using param.asInt().
Why is the relay active-LOW?	The selected relay module is assumed to energize when its input is LOW. The actual board must be verified before connecting the pump.
Why use a separate pump supply?	A GPIO pin cannot supply motor current. A separate rated DC source powers the pump through relay contacts, protecting the controller supply path from load current.
What does BlynkTimer do?	It schedules soilMoistureSensor() every 1000 milliseconds without placing repeated cloud writes directly inside the main loop.
Why call Blynk.run() repeatedly?	It services Blynk communication and callbacks. Long blocking delays can make cloud communication less responsive.
What happens when Blynk connects?	BLYNK_CONNECTED() calls pumpOff() and writes zero to V1, establishing a safe OFF command and dashboard state.
What are the main limitations?	No automatic threshold control, no offline fallback, no maximum pump runtime, no reservoir-level interlock and no experimentally validated calibration or water-saving result are included in the current code.
What would you implement next?	Validate sensor calibration, complete safe pump integration, add runtime protection and sensor-fault handling, then develop a tested automatic mode with hysteresis and manual override.

During evaluation, distinguish a feature that is present in the code from a proposed improvement. Report an experiment as completed only when the team has recorded the procedure, observations and evidence.

APPENDIX E · BLYNK IOT CLOUD IMPLEMENTATION

E.1 Cloud provisioning workflow

A reproducible sequence for connecting the ESP8266 to the Blynk IoT platform.

The Blynk workflow begins in the cloud console. A template defines the general device type and its data interface; a device instance represents the particular physical NodeMCU used by the project. The firmware is compiled with the template identifier, template name and device authentication token. The token identifies the device during connection and must be treated as a secret.

Recommended configuration sequence

25. Create or open the project template for the Smart Plant Watering System and confirm that the device type is appropriate for ESP8266 firmware.
26. Create a device from the template. Record the device name and template association in the private project handover notes.
27. Create the V0 datastream as an integer value with a minimum of 0 and maximum of 100. Label it as a moisture indicator or moisture percentage estimate.
28. Create the V1 datastream as an integer or switch-compatible value with 0 representing OFF and 1 representing ON.

29. Generate or copy the device authentication token into the private firmware source. Do not place the token in a public report, slide deck or repository.
30. Confirm the dashboard widgets use the same virtual pins and compatible data types as the firmware.
31. Upload the firmware and observe the device connection state before attempting pump control.

The firmware connects to the host blynk.cloud on port 80 through Blynk.begin(). The device also needs working Wi-Fi and Internet access. A successful Wi-Fi association does not by itself prove that Blynk authentication and device-template association are correct.

Configuration record

The report intentionally omits the live device token and Wi-Fi credentials. Keep a private record of the device name, template name, datastream settings and credential rotation date.

Provisioning and acceptance record

A device should not be considered provisioned merely because a template exists. The device instance must be associated with the intended template, the firmware must contain the matching template identifiers, and the authentication token must belong to that device. If the token is regenerated, update the private firmware copy and upload the new build. A mismatch between template, device and token can produce connection failures even when the Wi-Fi credentials are correct.

For a repeatable handover, record the date of device creation, the template name, the device label, the configured host, the port and the datastream names. Do not record the secret token itself in this public document. Instead, record where the authorized team member stores it securely and who is allowed to rotate it.

APPENDIX F · BLYNK DATA CONTRACT

F.1 Virtual-pin and dashboard data specification

A data contract defines what each software channel means, which side writes it and what value range is valid. This prevents a common integration error in which a dashboard widget is connected to a virtual pin that the firmware never reads or writes. Virtual pins are logical Blynk channels; they are not physical ESP8266 GPIO numbers.

Item	V0 moisture telemetry	V1 manual pump command
Direction	Device to cloud/dashboard	Dashboard/cloud to device
Firmware API	Blynk.virtualWrite(V0, moisture)	BLYNK_WRITE(V1) callback
Data meaning	Mapped moisture indicator	Requested pump state
Expected range	Integer 0 to 100	Integer 0 or 1
Dashboard widget	Gauge, value display or chart	Switch widget
Update rule	Scheduled every 1000 ms when connected	Sent when user changes switch
Physical relation	Derived from A0 reading	Callback operates D4 relay output

Dashboard configuration checks

- Set the V0 widget minimum and maximum to match the 0–100 range used in firmware.
- Add a percent-style label only as a display convention; do not imply laboratory-certified VWC.
- Configure the V1 widget to send 1 for ON and 0 for OFF.
- Verify the dashboard switch resets to OFF when the firmware executes BLYNK_CONNECTED().
- Use a clear device label so that screenshots can be attributed to the correct project device.

A dashboard screenshot should show the project device identity, V0 display and V1 switch. If the dashboard includes historical charts, record the time range and sampling interval used. The current sketch sends a fresh V0 value at each one-second timer event only while Blynk.connected() is true.

Data validation and dashboard usability

The dashboard should make the state understandable without requiring the user to interpret raw ADC counts. A descriptive label such as “Soil moisture indicator” is preferable to a generic “Value” label. The V1 switch should have an obvious ON/OFF state and should be tested after reconnecting the device, because the firmware explicitly writes zero to V1 in BLYNK_CONNECTED(). The dashboard must not imply that the pump is running solely because the switch state is ON; a future pump-current or flow sensor would be required to confirm actual operation.

When reviewing a screenshot, verify that the displayed value is current, the device identity is visible and the datastream has the expected range. A static dashboard image can demonstrate configuration, but it cannot prove one-second update timing or reliable command response. Those claims require a timestamped log or a short recorded test.

APPENDIX G · ESP8266 INPUT AND SIGNAL INTEGRITY

G.1 ADC range, wiring quality and measurement stability

The analog input is the boundary between the sensor electronics and the microcontroller. Before connecting the sensor output to A0, verify the exact NodeMCU board revision and its input divider. The ESP8266EX chip-level ADC input is not necessarily identical to the voltage range exposed at the development board connector. Applying an excessive voltage can damage the input even when the firmware itself is correct.

Practical measurement checks

Check	Why it matters	Recommended action
A0 input range	Board variants may scale the chip ADC differently.	Check the board schematic or manufacturer specification.
Sensor supply	Supply variation can shift the analog output.	Use a stable, module-compatible regulated supply.
Ground reference	A floating or poor reference causes unstable readings.	Use the grounding arrangement specified for the sensor and board.
Cable routing	Motor wires can inject switching noise.	Keep sensor wiring away from pump leads and relay contacts.
Probe movement	Changing insertion depth changes the measured soil region.	Mark and maintain a repeatable insertion depth.
Sampling variation	Single readings can fluctuate.	Record a series of samples and compare median or mean values.

The final firmware samples one reading during each timer callback. It does not average a group of ADC samples. A later revision could collect several samples, reject outliers and report a filtered value. Filtering should be selected carefully: excessive smoothing can hide rapid changes or delay a response, while too little filtering can make the dashboard fluctuate.

Board-specific caution

Do not assume every board labelled NodeMCU exposes the same A0 voltage range. Confirm the actual board before connecting the sensor output.

Noise and signal conditioning

The sensor output should be checked with the pump switched OFF and again during relay switching. If the raw value changes sharply when the relay or motor operates, the cause may be supply disturbance, shared wiring impedance or electromagnetic coupling. Separate routing for sensor leads and motor leads, a stable supply and

appropriate suppression at the motor/driver can reduce interference. Any change to grounding must follow the relay module and controller requirements; do not remove a required reference connection merely to reduce noise.

A useful baseline is a time series of raw readings with the probe held in an unchanged sample. Calculate the minimum, maximum and spread. Repeat the same observation after the pump is switched several times. If the spread changes materially, investigate the electrical layout before treating the fluctuation as a real change in soil moisture.

APPENDIX H · CALIBRATION MATHEMATICS

H.1 Worked examples using the submitted endpoints

The firmware uses DRY_VALUE = 1023 and WET_VALUE = 450. The Arduino map() operation scales the raw input from the interval [1023, 450] into [0, 100]. Because the input endpoints are in descending order, the displayed indicator increases as the raw ADC reading moves from the dry endpoint toward the wet endpoint.

Worked examples

Raw ADC value	Approximate mapped value	Interpretation under code assumptions
1023	0%	At the configured dry endpoint
880	25%	Closer to the dry endpoint
736 or 737	50%	Approximately halfway between endpoints
593	75%	Closer to the wet endpoint
450	100%	At the configured wet endpoint

For a raw value of 736, the linear expression gives ((1023 - 736) / (1023 - 450)) × 100, which is approximately 50.1%. The integer map() implementation returns an integer, so rounding and truncation behaviour should be considered when comparing the displayed value with a hand calculation.

Why constrain() is used

A sensor can produce readings slightly beyond the chosen endpoints because of noise, supply variation or calibration drift. constrain(moisture, 0, 100) limits the display value to the defined range. This prevents negative values or values above one hundred from appearing on the dashboard. It does not make the underlying sensor measurement more accurate; it only bounds the displayed number.

Interpretation boundary

The mapped result is a relative indicator derived from two calibration points. It should not be reported as true volumetric water content without a soil-specific reference calibration.

Worked interpretation of out-of-range readings

If the raw ADC reading is below 450, the unconstrained linear calculation would produce a value above 100. constrain() clips the displayed result to 100. If the raw reading is above 1023, the unconstrained calculation would fall below zero and the display is clipped to 0. These boundary behaviours protect the dashboard presentation, but repeated out-of-range readings should trigger a calibration or wiring review rather than being ignored.

A midpoint calculation is a numerical illustration only. The same raw ADC value can correspond to different actual moisture conditions in different soils or probe installations. The team should keep the endpoint pair with the soil type, probe depth and supply conditions under which it was measured so that the indicator can be reproduced.

APPENDIX I · SENSOR UNCERTAINTY AND CALIBRATION QUALITY

I.1 Sources of measurement variation

A moisture probe does not measure the entire pot or field. It responds to the local region around its sensing elements. The same plant can have moisture gradients caused by drainage, sunlight, root density, soil compaction and the position of the water outlet. A single sensor reading is therefore a sample of one location rather than a complete description of the root zone.

Factors that influence the reading

- Soil texture and bulk density change the relationship between electrical properties and water content.
- Salinity and dissolved minerals can change conductivity independently of water content.
- Temperature can affect probe and soil electrical characteristics.
- Probe depth and insertion pressure affect contact with the surrounding medium.
- Resistive probes may corrode or change response over repeated wetting cycles.
- Supply voltage and wiring resistance can introduce offset or noise.

Improving repeatability

Use the same soil container, probe position and supply arrangement during calibration and routine use. Allow the reading to stabilize after moving or watering the probe. Collect multiple readings in each condition and retain the minimum, maximum and representative value. If the spread is large, investigate wiring, grounding, sensor condition and placement before changing the calibration constants.

A useful calibration record contains raw readings, not only the final endpoint constants. Keeping the raw values allows the team to revisit the choice of endpoints later and compare the sensor response after maintenance or replacement. For a high-confidence VWC result, the sensor must be calibrated against a suitable reference method for the target soil.

Distinguishing sensor drift from soil change

A slowly changing dashboard value can reflect a real drying or wetting process, but it can also reflect probe corrosion, changing contact pressure, temperature or supply variation. To distinguish these effects, compare the sensor reading with a controlled reference sample and inspect the probe physically. If the same reference condition produces a different reading after several days or weeks, the calibration may have drifted.

A second probe placed at another location can help reveal spatial variation, but it also introduces the need to calibrate each probe. A multi-sensor system should not assume that all sensors share identical endpoints. Store the calibration constants per node or per sensor and document when a sensor is replaced.

APPENDIX J · ELECTRICAL DESIGN CONSIDERATIONS

J.1 Relay selection and pump load sizing

The relay module is an interface between the low-voltage controller signal and the pump power circuit. The NodeMCU GPIO does not drive the motor directly. Instead, D4 drives the relay module input, and the relay contacts switch a separate DC supply path to the pump. This distinction protects the GPIO from motor current, provided the selected module and wiring are appropriate.

Selection parameters to record

Parameter	What to verify
Pump rated voltage	The DC voltage printed on the pump or specified by its datasheet.
Pump running current	Current under the intended hydraulic load.
Pump startup/stall current	Transient current can exceed the normal running current.

Parameter	What to verify
Relay contact rating	DC rating at the relevant voltage and inductive load category.
Relay coil/module supply	Voltage and current required by the relay board.
Protection	Fuse, reverse-polarity protection and transient suppression where appropriate.
Wire and connector rating	Current capacity, insulation and secure mechanical termination.

A relay marked with a large AC current rating must not automatically be assumed to support the same DC motor current. DC arcs are harder to interrupt than AC arcs in many contact arrangements. Use the manufacturer's DC contact rating and include motor startup current in the selection decision. The pump supply must be independently rated and must not be taken from a NodeMCU GPIO pin.

Pre-energization rule

Verify the relay truth table and pump supply with the motor disconnected first. Only connect the pump after the control output, contact path, voltage, polarity and current ratings have been checked.

Contact and transient considerations

A DC motor is an inductive load and may generate transient voltage when current is interrupted. Use the pump or driver manufacturer's recommended suppression method, which may include a diode for an appropriate DC motor arrangement or another specified transient suppressor. Do not add a protection component blindly: polarity, switching topology and component ratings matter. Check that the relay module input is compatible with the ESP8266 logic level and that the relay board supply arrangement matches its documentation.

For a more complete electrical record, note the pump nameplate or datasheet ratings, measured supply voltage under load, measured running current and any observed startup behaviour. The relay should be selected using the actual DC load conditions, not just the largest number printed on a generic module listing.

APPENDIX K · HYDRAULIC AND MECHANICAL INTEGRATION

K.1 Pump placement, tubing and water delivery

Electrical switching is only one part of irrigation. The pump must move water from a reservoir to the plant through a tube or outlet. The hydraulic arrangement affects whether the plant receives a useful and repeatable amount of water. A pump may run but deliver little water if the lift height is too large, the tubing is kinked, the inlet is blocked or the reservoir level is too low.

Mechanical design checks

- Confirm the pump is suitable for the intended liquid and continuous or intermittent duty cycle.
- Place the pump inlet below the minimum operating water level if the design requires submersion.
- Secure tubing to prevent it from slipping, kinking or spraying water toward electronics.
- Keep the controller, relay and connectors above the reservoir and outside the splash zone.
- Check that the outlet does not create a concentrated erosion channel or bypass the root zone.
- Provide a stable base for the pot and reservoir so that accidental movement does not pull on wires.

Measuring delivered water

If pump performance is to be evaluated, measure delivered volume over a known interval using a graduated container or mass measurement. Repeat the measurement at the intended lift height and tubing configuration. Record the supply voltage and test duration. A nominal pump flow rating from a product listing may not represent the flow achieved in the assembled system because the actual head, tubing and restrictions affect delivery.

The current firmware does not measure flow or confirm that water was delivered. A future design could add a flow sensor or reservoir-level switch so that the dashboard can distinguish a commanded pump state from a verified hydraulic outcome.

Reservoir and delivery safeguards

The reservoir should be large enough for the planned demonstration and positioned so that its water level is visible. If the pump is not designed to run dry, the user should not leave it energized when the reservoir is empty. A future low-level switch can inhibit pump operation, while a runtime limit can stop the pump if a tube is blocked or the dashboard command is left ON. A check valve or suitable routing may be required to prevent unwanted siphoning after the pump stops.

The outlet should distribute water near the intended root zone without flooding the pot or running water directly onto the sensor electronics. During commissioning, inspect the entire path from reservoir to outlet and then observe the soil response. A command acknowledgement is not equivalent to proof of successful irrigation; measuring delivered volume or observing water flow provides stronger evidence.

APPENDIX L · COMMISSIONING AND SAFE STARTUP

L.1 Commissioning gates before integrated operation

Commissioning is the controlled transition from separate component checks to an integrated demonstration. The sequence below is designed to prevent the pump from being energized before the sensor input, relay logic and supply path are understood. Keep the pump disconnected until the relay output has been verified.

Gate	Acceptance check	Evidence to retain
G1 - Board	NodeMCU powers and uploads firmware reliably.	Board identification and serial startup log.
G2 - Sensor	A0 raw reading changes in the expected direction.	Dry/wet sample table and sensor photo.
G3 - Relay	D4 HIGH and LOW produce the expected relay state.	Observation with pump disconnected.
G4 - Cloud	Device connects and V0 updates.	Blynk device status and dashboard capture.
G5 - Command	V1=0/1 changes D4 as intended.	Callback log and relay observation.
G6 - Load	Pump starts/stops safely from a suitable DC supply.	Electrical readings and supervised video.
G7 - Water path	Water reaches the intended area without leaks.	Leak check and delivered-volume record.

Each gate should be signed off before proceeding to the next. If a gate fails, return to the component or subsystem check rather than changing several parts at once. This makes faults easier to isolate and reduces the chance of damaging the controller or pump.

Safe demonstration

Use a low-voltage DC pump supply selected for the actual pump. Keep water away from exposed electrical connections and conduct the integrated test under supervision.

Commissioning sign-off record

For each gate, record the responsible team member, date, instrument used and the evidence filename. A gate should be marked accepted only when the expected observation is present. If the test is skipped, mark it pending. If the observed behaviour differs from the expected behaviour, record the discrepancy and do not continue to the next riskier stage until the cause is understood.

The commissioning record should be stored with the firmware version used for the test. A change to the relay pin, calibration endpoints, relay polarity or Blynk datastream configuration creates a new test baseline and requires the related checks to be repeated.

APPENDIX M · DETAILED FUNCTIONAL TEST PROTOCOL

M.1 Step-by-step test procedure

Test 1 - Boot and safe default

Disconnect the pump load, connect the NodeMCU to a current-limited USB supply and open the Serial Monitor at 9600 baud. Reset the board and confirm the startup message. Verify that setup() configures D4 as an output and calls pumpOff(). Observe the relay module and record whether it remains de-energized during boot and after initialization.

Test 2 - Sensor response and mapping

Record raw ADC readings in the chosen dry and wet reference conditions. Confirm that the wet condition produces a lower reading than the dry condition under the assumptions in the current firmware. Calculate the mapped indicator and compare it with the Serial Monitor and V0 dashboard. Repeat the measurement to observe variation.

Test 3 - Cloud telemetry

Confirm that the Blynk device is online. Observe V0 over several timer intervals. The expected update schedule is approximately once per second while connected. Record any missed updates, reconnects or long periods without telemetry rather than assuming the nominal timer interval is the measured cloud latency.

Test 4 - Manual relay control

With the pump still disconnected, set V1 to 1 and observe D4 LOW and relay energization. Set V1 to 0 and observe D4 HIGH and relay de-energization. Repeat at least five cycles and record any inconsistent switching. Only after relay behaviour is verified should the DC pump circuit be connected.

Test 5 - Reconnect and load integration

Reconnect Blynk and verify the callback commands OFF and writes zero to V1. Then, under supervision, test the pump using the rated DC supply and check for leaks, heating, abnormal noise and secure tubing. Record the result and evidence filename for every test.

Test evidence and repeatability

For each test, capture the initial condition, the action, the expected response and the observed response. A dashboard screenshot can support the V0 or V1 configuration claim; a Serial Monitor capture supports the raw ADC and mapped value claim; a short video can support the relay and pump observation. Use filenames that include the test identifier and date so the evidence can be traced to the report table.

Repeat switching tests several times rather than accepting a single successful event. If the pump is integrated, check that the supply voltage remains within the pump specification and that the relay contacts do not show abnormal heating or chatter. Stop immediately if there is leakage, unexpected movement, exposed wiring or an unusual smell or sound.

APPENDIX N · EXPERIMENTAL RECORD SHEETS

N.1 Sensor, dashboard and relay observation logs

The tables on this page are intended to be completed during physical testing. They are deliberately left as record fields rather than being populated with assumed measurements. Use the same units and test conditions throughout a measurement series.

Sensor calibration observations

Sample	Condition / notes	Raw ADC	Mapped %
1	Dry reference	To be recorded during the test.	To be recorded during the test.
2	Dry reference	To be recorded during the test.	To be recorded during the test.
3	Dry reference	To be recorded during the test.	To be recorded during the test.
4	Wet reference	To be recorded during the test.	To be recorded during the test.
5	Wet reference	To be recorded during the test.	To be recorded during the test.
6	Wet reference	To be recorded during the test.	To be recorded during the test.
7	Intermediate soil	To be recorded during the test.	To be recorded during the test.
8	Intermediate soil	To be recorded during the test.	To be recorded during the test.

Relay and dashboard observations

Trial	V1 command	Expected D4	Observed relay / pump	Pass / notes
1	0	HIGH	To be recorded during the test.	To be recorded during the test.
2	1	LOW	To be recorded during the test.	To be recorded during the test.
3	0	HIGH	To be recorded during the test.	To be recorded during the test.
4	1	LOW	To be recorded during the test.	To be recorded during the test.

Record the test date, tester, board and module identifiers, pump supply rating and any deviation from the planned setup. Attach serial logs and original photographs as supporting evidence.

Cloud connection observation log

Time / trial	Device status	V0 displayed value	V1 command	Notes
1	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.
2	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.
3	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.
4	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.

Use the observation log to compare what the device reports with what the dashboard displays. Note any difference between device-side serial output and the cloud value. The table is a blank experimental form, not a results table; complete it only with readings captured from the actual system.

APPENDIX O · EVALUATION METRICS AND REPORTING

O.1 What to measure in a future performance study

The project is currently a functional architecture and firmware demonstration. A performance study should define measurable indicators before collecting data. This avoids making broad claims from a few dashboard screenshots or from code inspection alone.

Metric	Operational definition	How to measure
Sensor repeatability	Spread of readings under unchanged conditions.	Collect repeated ADC samples at fixed probe position.
Calibration fit	Agreement between mapped indicator and reference moisture.	Compare against gravimetric or other suitable soil reference.
Telemetry interval	Time between successive V0 updates.	Timestamp dashboard or device-side logs over a defined period.
Command response	Time from V1 change to relay state change.	Record synchronized command and output timestamps or video.
Pump delivery	Volume delivered in a known time.	Collect water in a graduated vessel at target head.
Power consumption	Current and energy used by controller and pump.	Measure separately using suitable meters.
Water saving	Difference from a defined baseline under matched conditions.	Compare controlled watering trials with a documented baseline.

Experimental comparison design

To estimate water savings, define a baseline schedule and a comparison schedule, use equivalent containers or plots, keep plant type and environmental conditions as consistent as possible, and measure water volume over the same observation period. Replicate trials and report variability. The current manual-control firmware is not itself proof of a water-saving percentage; any such result would require a separate controlled experiment.

Reporting rule

Do not claim a percentage reduction, sensor accuracy, latency or flow rate unless the underlying measurements and method are included in the report.

Recommended reporting format

A results section should state the setup, method, number of repetitions, measured value, variation and limitations. For example, a telemetry test should give the observation period and number of successful updates rather than only stating that the dashboard “worked.” A pump test should state the supply, pump model, test duration and measured or observed delivery. A calibration result should include the soil, probe position and raw endpoint samples.

If the data set is small, report it as a prototype observation and avoid generalizing to all soils or plants. A water-saving comparison requires a baseline system, comparable plants or plots and a defined period. Report the water volume for each condition and explain how environmental differences were controlled.

APPENDIX P · OPERATION AND MAINTENANCE

P.1 Routine inspection and service plan

A connected plant-watering prototype requires periodic inspection even if the firmware continues to run. Moisture probes can shift or degrade, water tubing can clog, reservoirs can empty and network credentials can

change. A simple maintenance plan helps preserve measurement consistency and reduces the chance of unexpected pump operation.

Interval / event	Inspection task	Record
Before each demonstration	Check water level, tubing, exposed wires and enclosure position.	Visual inspection and date.
Weekly during use	Inspect probe surface, connector tightness and sensor placement.	Probe condition and raw reading.
After changing soil or pot	Repeat calibration checks because the medium has changed.	New endpoint samples.
After Wi-Fi/router change	Reconfirm SSID, connectivity and Blynk device online state.	Connection check.
After token exposure	Rotate the token and update the private firmware.	Rotation date and access review.
After pump replacement	Recheck supply voltage/current and relay contact suitability.	Pump and relay ratings.
After a leak or fault	Disconnect power safely, dry and inspect before restart.	Fault cause and corrective action.

A maintenance record should distinguish an observed issue from a suspected cause. For example, a lower V0 value does not prove the soil contains a particular VWC; it may also reflect a shifted probe, changed salinity or altered sensor supply. Recheck the raw ADC value and physical installation before revising calibration constants.

Maintenance log template

Date	Inspection / change	Observation	Action taken	Initials
To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.
To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.
To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.	To be recorded during the test.

Keep a maintenance log when the sensor, relay, pump, power supply or firmware is changed. A change in one component can alter the response of another: a new sensor may need new endpoints, a new relay board may use different active logic, and a new pump may require a different supply or contact rating. The device should be recommissioned after any electrical change.

APPENDIX Q · FUTURE AUTOMATIC MODE DESIGN

Q.1 Proposed threshold control with hysteresis and safeguards

A future version may add automatic irrigation, but it should be implemented as a separately tested firmware mode. A single threshold can cause rapid ON/OFF switching when the reading fluctuates around the boundary.

Hysteresis uses separate start and stop limits: the pump starts only when the indicator is below a lower threshold and stops when it rises above an upper threshold.

Illustrative control policy (not implemented)

```
if (mode == AUTOMATIC) {
  if (sensorValid && !pumpIsOn && moisture < DRY_LIMIT) {
    if (reservoirHasWater) startPump();
  }
  if (pumpIsOn && moisture > WET_LIMIT) stopPump();
  if (pumpRuntime > MAX_RUNTIME) stopPump();
  if (!sensorValid || faultDetected) stopPump();
}
// Require DRY_LIMIT < WET_LIMIT for hysteresis.
```

Safeguards required before deployment

- Validate the sensor and select lower and upper limits from plant- and soil-specific experiments.
- Set a maximum pump runtime and a minimum rest interval.
- Use a reservoir-level interlock to prevent dry running.
- Detect disconnected or implausible sensor readings and command the pump OFF.
- Define what happens during Wi-Fi and cloud loss; local safety should not depend only on the dashboard.
- Retain a manual override and make the active control mode visible to the user.

Implementation status

The pseudocode is a design proposal. The submitted firmware does not contain a threshold comparison or automatic pump decision.

Mode transition and state ownership

When automatic mode is introduced, the firmware must define which source has authority over the pump. A simple policy is to make the selected mode explicit: MANUAL or AUTOMATIC. In manual mode, V1 controls the output. In automatic mode, the moisture controller controls the output and the dashboard displays the current mode and commanded state. A manual override should have a defined timeout or a clear rule for returning to automatic operation.

The system should also distinguish commanded state from measured state. The boolean Relay variable in the current code records the state commanded by the helper functions; it does not confirm that the relay contacts moved or that the pump delivered water. A future feedback input such as current sensing, flow sensing or a contact monitor would provide additional evidence of actuation.

APPENDIX R · TEAM HANDOVER AND EXTENDED VIVA

R.1 Team contributions and evaluation questions

Team contribution summary

Member	Registration no.	Contribution summary
Anuj Parihar	20247015	Case-study framing, scalability analysis, report structure and technical documentation.
Aman Kharwar	20247010	Hardware interfacing, sensor/relay/pump planning and firmware implementation.
Mayank Gupta	20247027	Blynk IoT Cloud configuration, device setup and integration support.
Sudarshan Jha	20247051	Problem analysis, system planning and technical research.

Additional viva questions

Question	Answer
Why is V0 not A0?	A0 is a physical analog input. V0 is a logical Blynk virtual pin used to transmit the calculated indicator.
Why is V1 not D4?	V1 carries the dashboard command. The callback translates that command into a physical output on D4.
Why use BlynkTimer?	It schedules periodic sensor acquisition without a blocking delay in loop().
What is the purpose of constrain()?	It limits the mapped indicator to the 0–100 display range.
Does BLYNK_CONNECTED() make the pump automatic?	No. It commands the pump OFF and resets the V1 dashboard value when a connection is established.
What is the main network dependency?	Remote display and commands depend on Wi-Fi, Internet access and the Blynk Cloud connection.
How can calibration be improved?	Collect repeated dry/wet readings in the target soil and compare with an independent reference.
What prevents GPIO damage from the pump?	The relay interface switches a separate pump supply; the pump current must not flow through the GPIO.

The team should be prepared to explain which features are implemented, which have been physically verified and which are future work. The most defensible answer is always the one supported by the current source code and recorded test evidence.

Questions about project scope and evidence

Question	Answer
What does the project demonstrate today?	Automatic acquisition and cloud reporting of the moisture indicator, plus user-initiated remote pump switching through a relay.
What evidence is still needed?	Original hardware photographs, measured calibration samples, V0 telemetry records, V1 relay tests and supervised pump observations.
Why not call the displayed value true VWC?	The current code uses a two-point linear mapping; it has not been calibrated against a soil-specific volumetric reference method.
Why can the pump remain ON if the network fails?	The current code has no explicit offline stop timer or local fallback policy. This is a known limitation to address before unattended use.

R.2 Recommended final demonstration sequence

1. Begin with the pump disconnected from the relay contacts. Power the NodeMCU from a regulated supply, open the Serial Monitor at 9600 baud and confirm that the firmware starts with the pump commanded OFF.
2. Insert the moisture probe into the test medium at a repeatable depth. Observe the raw ADC reading and the mapped indicator. Record dry, wet and intermediate observations in the laboratory record; do not treat the code constants as measured calibration results.
3. Confirm that the Blynk device is online and that the V0 dashboard value updates periodically. Compare the displayed value with the Serial Monitor output and note any delay, missing update or unexpected jump.
4. Test the V1 manual switch first with the pump disconnected. Confirm that V1 = 1 commands D4 LOW and V1 = 0 commands D4 HIGH. Only after confirming the relay polarity and contact wiring should the low-voltage DC pump supply be connected for a supervised functional check.
5. Reconnect the device to Blynk and verify that the connection callback commands the pump OFF and writes zero to V1. Record the observed behaviour during a network interruption; the current firmware does not provide a separate offline automatic-control mode.

R.3 Evidence to retain during handover

- Original photographs of the actual NodeMCU, sensor, relay, pump, supply and completed wiring; label conceptual illustrations separately.
- Blynk Console screenshots showing the device, V0 moisture datastream, V1 manual-control datastream and the live dashboard without exposing the device authentication token.
- Serial Monitor captures showing raw ADC values and mapped moisture values, together with the measured dry and wet calibration endpoints and the soil or medium used.

- A supervised relay and pump test record containing the pump supply rating, relay contact rating, observed ON/OFF behaviour, water-delivery observation and any faults found.

R.4 Extended technical viva: firmware and hardware

How is the moisture indicator calculated?

The firmware maps the raw A0 reading between DRY_VALUE = 1023 and WET_VALUE = 450 into a 0–100 range. The map() function performs the linear scaling, and constrain() clips the result to the display range.

Why are the dry and wet calibration values not universal?

The endpoints depend on the sensor board, supply, soil composition, salinity, probe depth and installation. They must be measured for the actual setup; the values in the source code are provisional calibration constants.

What happens if the raw reading falls outside the selected endpoints?

The linear map can produce a value outside the intended interval. constrain() limits the displayed value to 0–100, but clipping does not correct poor calibration or diagnose a disconnected or faulty sensor.

Why is BlynkTimer used instead of a long delay?

BlynkTimer schedules sensor acquisition at a 1000 millisecond interval while loop() continues to call Blynk.run(). This allows the firmware to service cloud communication and callbacks between sensor updates.

What does the Relay Boolean variable represent?

Relay records the state commanded by pumpOn() or pumpOff(). It is not an independent electrical feedback signal and does not prove that the relay contacts moved or that water actually flowed.

Does the soil moisture routine switch the pump automatically?

No. soilMoistureSensor() reads A0, maps and prints the value, and sends the indicator to V0 when Blynk is connected. Pump commands are handled separately by BLYNK_WRITE(V1).

What happens when the device connects or reconnects to Blynk?

BLYNK_CONNECTED() calls pumpOff() and writes zero to V1. This establishes a commanded OFF state after a successful connection; physical relay behaviour must still be verified during testing.

What is the significance of using D4 on a NodeMCU board?

D4 is a board label that commonly maps to GPIO2. ESP8266 boot behaviour can depend on GPIO levels, so the selected relay module must not force an invalid boot level. Verify startup with the pump disconnected.

Why must the A0 input voltage be checked before wiring?

The usable analog input range depends on the specific ESP8266 development board and its onboard divider. Confirm the board documentation and sensor output voltage before connecting A0 to avoid exceeding the input rating.

How should a future automatic mode be introduced safely?

Use separate dry and wet thresholds to provide hysteresis, a maximum pump runtime, a rest interval, a reservoir-level interlock and sensor-fault handling. Keep a local stop condition so pump safety does not rely only on cloud connectivity.